BE Interim Report

Solving Optimisation Problems on Annealing-Type Quantum Computers

Declaration of Authorship

I have read and understood the university's policy on plagiarism. I declare that all material in this assessment is my own work except where there is clear acknowledgement and appropriate reference to the work of others.

Signed: Lachlan walker

Date: 18/01/2026

Contents

Acronyms 3
1. **Introduction** 4
1.1 Background and Motivation 4
1.2 Project Aims 5
1.3 Scope of the Project 5
1.4 Report Structure 5
2. **Literature Review** 6
2.1 Combinatorial optimisation 6
2.2 Classical techniques for optimisation 7
2.3 Quantum Annealing and Quantum-inspired methods 7
2.4 Formulations of QUBOs 8
2.5 Research Gap to be Addressed 10
3. **Work Completed to Date** 11
3.1 Overview of Work Completed 11
3.2 Problem Selection and Dataset 11
3.3 QUBO Formulation 12
3.4 Solver Implementation 13
3.5 Preliminary Results 14
3.6 Challenges and Limitations 16
4. **Future Work** 17
5. **Ethics and Sustainability** 18
References 19

Acronyms

COPs combinatorial optimisation problems

QUBO Quadratic unconstrained binary optimisation

SDK Software Development Kit

SA Simulated Annealing

QA Quantum Annealing

SQA Simulated Quantum Annealing

ME Minor Embedding

BQM Binary Quadratic Model

Chapter 1

Introduction

1.1 Background and Motivation

Combinatorial optimisation problems are some of the most common computational problems facing plenty of fields such as logistics, finance, telecommunications, scheduling and machine learning. These problems involve selecting an optimal combination of decisions from a large search space following some given constraints. The nature of these problems leads to severe computational complexity for large problem instances. One well-studied formulation for these problems is Quadratic unconstrained binary optimisation (QUBO), where the objective is expressed as a quadratic function over binary variables. QUBO problems are NP-hard in general. They can also be shown to encode a variety of combinatorial optimisation problems[1].

QUBO problems can be solved using a variety of optimisation algorithms. One of the best-established heuristics is simulated annealing (SA), based on the process of cooling metal, which can be applied to find approximate solutions for challenging COPs. Quantum annealing (QA) is another heuristic method that can exploit quantum mechanical effects to explore the solution space efficiently. QA tries to find a system's lowest energy state by gradually evolving a quantum system's Hamiltonian. This process can, in theory, escape local minima more effectively than classical methods.

Recently, studies have compared quantum annealing, simulated annealing, and hybrid quantum-classical solvers at solving QUBO instances of the NP-hard Max-Cut problem and analysed trade-offs between solution quality and computational efficiency[2].

1.2 Project Aims

This project aims to investigate the performance of classical and quantum methods on QUBO formulations of difficult real-world combinatorial optimisation problems. The project focuses on transforming our selected problems into the standard QUBO form and solving them using both classical methods, such as simulated annealing and quantum or quantum-inspired methods, such as simulated quantum annealing. The primary performance metric of interest is runtime, with additional considerations of accuracy and scalability across the different problem instances.

1.3 Scope of the Project

The scope of this project is limited to the benchmarking of these optimisation methods on QUBO instances. To date, work has been completed on Max-Cut problems with up to 90,000 variables using D-Wave's Ocean SDK in Python. Future work extends to alternative problem types and QUBO encodings. This project does not yet include any real-world case studies from industry; the findings are intended to give insight into these methods rather than claim a quantum advantage.

1.4 Report Structure

The remainder of the report is structured as follows: Section 2 provides a literature review of General combinatorial optimisation solution techniques, classical solution methods, quantum solution methods and converting problems into the QUBO form. Section 3 provides a description of the work completed so far, including methods and preliminary results. Section 4 outlines the future work to be completed in a project plan. Section 5 discusses any ethics and sustainability issues. Finally, the references cited in this report are listed at the end.

Chapter 2

Literature Review

2.1 Combinatorial optimisation

In general, a combinatorial optimisation problem is expressed as the following objective:

$$\min_{x \in X} f(x)$$

Where $X \subset \{0,1\}^n$ is a discrete set and f(x) is the objective function to be minimised.

Combinatorial optimisation problems involve choosing the optimal set of choices from a discrete set of choices. These problems arise across many fields such as logistics, finance, telecommunications, scheduling and machine learning. Initially, it may seem that solving a discrete variable problem would be easier than a continuous variable problem. For a variable within a given range, a set of discrete values within the range is finite, whereas the number of continuous values is infinite; it seems it would be easier to search from a finite set rather than from an infinite set. However, this is not the case; the smoothness of the continuous functions means the objective function and constraint function values at a point x can be used to deduce information about points in a neighbourhood of x. Solving discrete problems involves finding the best solution among finite, but often astronomically large possibilities. Many problems in this area are NP-hard, making exact solution methods such as exhaustive search infeasible for large problem instances[3].

Due to this, heuristic methods become more reliable methods for solving large problems. The lack of standardisation in how combinatorial optimisation problems and metaheuristics are formulated makes it difficult to classify and compare metaheuristics' behaviours and performance[4].

2.2 Classical techniques for optimisation

Classical approaches to COPs usually rely on heuristic or metaheuristic methods, which trade the guarantee of optimality for scalability and practicality. Among many methods, simulated annealing is one of the best-established methods and has been applied to a wide range of COPs[5].

Simulated annealing was inspired by the process of physical annealing in metals; it operates probabilistically by accepting or rejecting possible solutions. Given some change in the objective value $\Delta \mathrm{E} = E_{new} - E_{old}$ the probability of acceptance is:

$$P(\Delta \mathrm{E}) = \begin{cases} 1, & \Delta \mathrm{E} < 0 \\ exp\left(-\frac{\Delta \mathrm{E}}{T}\right), & \Delta \mathrm{E} > 0 \end{cases}$$

Where T is a temperature parameter that gradually reduces according to some cooling schedule. This allows for simulated annealing to escape local minima early in the search and motivates convergence later in the schedule. Simulated annealing can be ineffective when landscapes have steep barriers, particularly near low-energy states.

Simulated annealing's performance relies heavily on parameter tuning and the problem topology[6][7]. Studies have compared heuristics and concluded that no single classical method consistently outperforms the others across problem classes, which drives the need for benchmarking[8].

2.3 Quantum Annealing and Quantum-inspired methods

Quantum Annealing is an optimisation technique that can exploit quantum mechanical effects to find low-energy configurations of some problem Hamiltonian. The foundation of QA was introduced by Kadowaki and Nishimori [9], who considered replacing the thermal fluctuations with quantum fluctuations in order to navigate complex energy landscapes more effectively.

QA encodes the optimisation problem as a time-dependent Hamiltonian given as:

$$H(t) = A(t)H_D + B(t)H_P$$

Where H_D is the driver Hamiltonian inducing quantum fluctuations and H_P is the problem Hamiltonian whose ground state encodes the solution to the problem. $A(t)$ and $B(t)$ are time-dependent coefficients that satisfy $A(0) \gg B(0)$ and $A(T) \ll B(T)$. During the process of adiabatic evolution, the system remains in the ground state as the Hamiltonian transitions slowly from H_D to H_P[10].

Practical implementations of QA have been achieved, such as the quantum annealer from D-Wave Systems. These systems are often limited by qubit count, connectivity, noise and annealing times. Direct access to these state-of-the-art systems is limited; as a result, hybrid quantum-classical solvers and simulated quantum annealing (SQA) have been developed to emulate QA on classical hardware. Previous benchmarking studies have shown mixed results, with performance largely dependent on the problem structure and quality of the encoding[2][11][12].

2.4 Formulations of QUBOs

The QUBO model has emerged as a framework for embracing COPs in a form that is compatible with both classical and quantum solvers. QUBO problems can be defined as:

$$\min_{x \in X} x^T Q x$$

Where x is a vector of binary decision variables, and Q is a real-valued $n \times n$ matrix encoding the relationships and constraints between variables. Most real-world COPs include constraints that must be satisfied by a feasible solution. The nature of QUBO means that problem

constraints must be reformulated as penalty terms within the objective. For example, given a constraint $g(x) = 0$, the quadratic penalty term

$$\lambda g(x)^2$$

Is added to the objective, where $\lambda > 0$ is a penalty coefficient that is chosen to make violating this constraint unfavourable. This is commonly required in problems involving scheduling, routing and assignment. Beyond basic constraint handling, many problems require unique transformation techniques in order to fit into the QUBO framework. Glover [13] provides many of these techniques for some of the most common problems.

QUBO is mathematically equivalent to the Ising model, which is commonly used in QA, the equivalent Ising model can be achieved from the following transformation. Binary variables are converted to spin variables:

$$x_i = \frac{1 + z_i}{2}, \qquad z_i \in \{-1, +1\}$$

The QUBO objective can be written as the Ising Hamiltonian:

$$H_P(z) = \sum_i h_i \, z_i + \sum_{i<j} J_{ij} \, z_i z_j$$

Where h_i maps to the diagonal entries of the Q matrix and J_{ij} maps to the off-diagonal entries of Q. To implement the resulting Ising model on the topology of the annealer, we must perform minor embedding [14] on the problem representation. This process involves representing the problem as a graph, which shows which variables interact with each other. If any variable is connected to more other variables than the topology allows for, then that variable is represented by multiple qubits in the annealer. This process is highly complex, and the quality of this embedding plays a large role in the effectiveness of QA.

2.5 Research Gap to be Addressed

There is a lot of research on solving COPs using classical, purely quantum and quantum-inspired methods; however, some gaps remain. The results of benchmarking are often dependent on the problem used for benchmarking, solver configuration and the encoding of the problem. These factors can make it difficult to compare methods in a meaningful way.

Most existing studies test relatively small problems or use problem setups designed to work well with specific implementations of solvers or hardware. Few studies examine how solvers perform on very large problems under real-world memory and computing limits. This matters because real-world COPs typically produce large problem formulations.

Additionally, simulated quantum annealing and hybrid solvers are often used as stand-ins for actual quantum hardware, but they don't face the same challenges that physical quantum annealers do, such as limited qubit connectivity and embedding quality. As a result, it's unclear when quantum-inspired methods provide practical benefits over classical approaches.

These gaps explain the focus of this project. By testing simulated annealing and simulated quantum annealing across different problems using consistent problem formulations, this work aims to provide clearer insights into how well solvers scale, how long they take to run, and how many resources they use. Rather than trying to prove a quantum advantage, the project focuses on understanding the practical trade-offs between solution quality and computational efficiency.

Chapter 3

Work Completed to Date

3.1 Overview of Work Completed

The primary focus of the work completed to date has been to establish a pipeline for benchmarking classical and quantum solutions to COPs that have been encoded in the QUBO framework.

To date, progress has been made in identifying appropriate benchmarking problems, formulating QUBOs and evaluating solver performance for both classical and quantum-inspired methods. Early tests have focused on the Max-Cut problem due to its simple QUBO formulation and widespread use as a benchmarking problem. Simulated annealing and simulated quantum annealing accessed from the D-Wave Ocean SDK were applied to these problem instances, and preliminary comparisons of runtime and solution quality were obtained.

3.2 Problem Selection and Dataset

The Max-Cut problem was selected as the initial focus due to its importance as an NP-hard COP and its popularity as a benchmarking problem. The Max-Cut problem seeks to, for some undirected graph $G(V, E)$ with V vertices and E edges, partition the vertices into two sets such that the number of edges between them (considered to be cut) is as large as possible. The edges of the graph can either be uniformly weighted or have some positive or negative weight, which influences how beneficial to the final energy an edge being cut is.

Benchmark instances for the Max-Cut problem were found from publicly available datasets[15]. The dataset contains 222 problem instances ranging from 1,000 to 90,000 variables, enabling investigation of both solvers' scalability and performance on large problem instances. A subset of the data was chosen with the weighting ranging from -1000 to +1000. At this early stage, emphasis was placed on identifying any practical limitations for solver runtime and resource usage. A summary of the dataset used in the benchmarking is provided below.

Number of Vertices	Number of edges	Number of Problem Instances	Weighting Range	Density
1000	2,964	15	{-1000,+1000}	5.93×10^{-3}
10,000	29,694	15	{-1000,+1000}	5.93×10^{-4}
20,000	59,394	15	{-1000,+1000}	2.97×10^{-4}
40,000	118,794	15	{-1000,+1000}	1.48×10^{-4}
90,000	267,294	14	{-1000,+1000}	6.6×10^{-5}

Table 3.1 summary of the dataset used in the benchmarking tests

3.3 QUBO Formulation

For the above problem instances, each was transformed into the QUBO framework following the process described in the literature[13]. Each of the vertices of the graph is represented by a binary decision variable that indicates which side of the cut the vertex belongs to. The objective is to assign vertices to either set such that the total weight of all edges deemed to be cut is maximised.

For efficiency, the Q matrix was stored in upper triangular form. In this representation, the diagonal entries Q_{ii} correspond to the number of edges connected to vertex i. These terms linearly contribute to the objective and reflect how each vertex i interacts with the rest of the vertices. The off-diagonal entries Q_{ij} for $i < j$ are equal to $-2w$ where w is the weight of the

edge between the vertices v_i and v_j, and are 0 when no such edge exists. This penalises possible solutions that place adjacent vertices in the same set, encouraging cuts with a higher total edge weight.

The Max-Cut problem can be directly formulated into the QUBO framework with no penalty terms required. The QUBO formulations of these problems were kept the same for solution by each method to ensure performance is less reliant on encoding quality.

3.4 Solver Implementation

Both classical and quantum-inspired solvers were applied to the same QUBO instances outlined above. Both SA and SQA solvers are available from the D-Wave Ocean SDK [16].

Classical SA was implemented as a Python function which takes the Q matrix as its input and outputs the runtime and best energy found. This was done by starting a timer using the time module in Python. Then the QUBO instance was set up as a Binary quadratic model (BQM), and the solver was set to the `SimulatedAnnealingSampler`. The solution was then found by sampling the model using a default temperature schedule. All parameter values were set to their default values, except the random seed, which sets the random starting point for the simulated annealing process; this was set to 37 for reproducibility across methods. Finally, the best energy is extracted from the solution and output. The timer is also stopped once a solution is found, and the runtime is calculated and output. The solution is performed by the code block:

```
bqm = dimod.BinaryQuadraticModel(Q, "BINARY")
sa_solver = SimulatedAnnealingSampler()
sa_solution = sa_solver.sample(bqm, num_reads=1, seed=37)
```

Quantum-inspired SQA was implemented as a similar function in Python, which takes the Q matrix as its input and outputs the runtime and best energy found. This was again done by starting a timer using the time module in Python. Then the QUBO instance was set up as a

BQM, and the solver was set to the `RotorModelAnnealingSampler`. Another solver that implements SQA is the `PathIntegralAnnealingSampler`. After testing, it was concluded that for this problem, the former achieved better solutions in less time. The solution was then found by sampling the model using default H_p and H_D fields. All parameter values were set to their default values, except the random seed, which was set to 37 for reproducibility.

Finally, the best energy is extracted from the solution and output. The timer is also stopped once a solution is found, and the runtime is calculated and output. The solution is performed by the code block:

```
bqm = dimod.BinaryQuadraticModel(Q, "BINARY")
sqa_solver = RotorModelAnnealingSampler()
sqa_solution = sampler.sample(bqm, num_reads=1, seed=37)
```

3.5 Preliminary Results

All 74 problem instances of the Max-Cut problem were solved by the SA and SQA solvers mentioned above, and the results were then averaged to create Table 3.2 which reports the average solution energy and average runtime for each problem size.

For small problem instances, both methods find solutions very quickly, but SA consistently finds better solutions than SQA. As problem size increases, the runtime of SA grows significantly, whereas SQA remains comparably efficient. For larger instances, SQA achieves runtimes around five times faster than those of SA, while producing solutions of comparable, though slightly lower, quality.

To examine how these trends may extend to larger problem instances, the data from Table 3.2 was visualised in Fig. 3.1. This highlights the rapid increase in runtime for SA as problem size increases, which indicates that scalability may become a limiting factor despite its marginally better solution quality.

Number of Variables	Method	Mean Energy (millions)	Std Energy (millions)	Mean Runtime (seconds)	Std Runtime (seconds)
1000	SA	0.46	0.02	0.09	0.002
1000	SQA	0.44	0.02	0.04	0.03
10000	SA	4.55	0.05	1.22	0.31
10000	SQA	4.29	0.05	0.34	0.08
20000	SA	9.08	0.09	2.62	0.63
20000	SQA	8.58	0.10	0.65	0.09
40000	SA	18.11	0.07	7.90	0.65
40000	SQA	17.08	0.07	1.69	0.29
90000	SA	40.72	0.20	26.87	0.54
90000	SQA	38.37	0.21	4.39	0.45

Table 3.2: Performance metrics for the solution of the Max-Cut problem

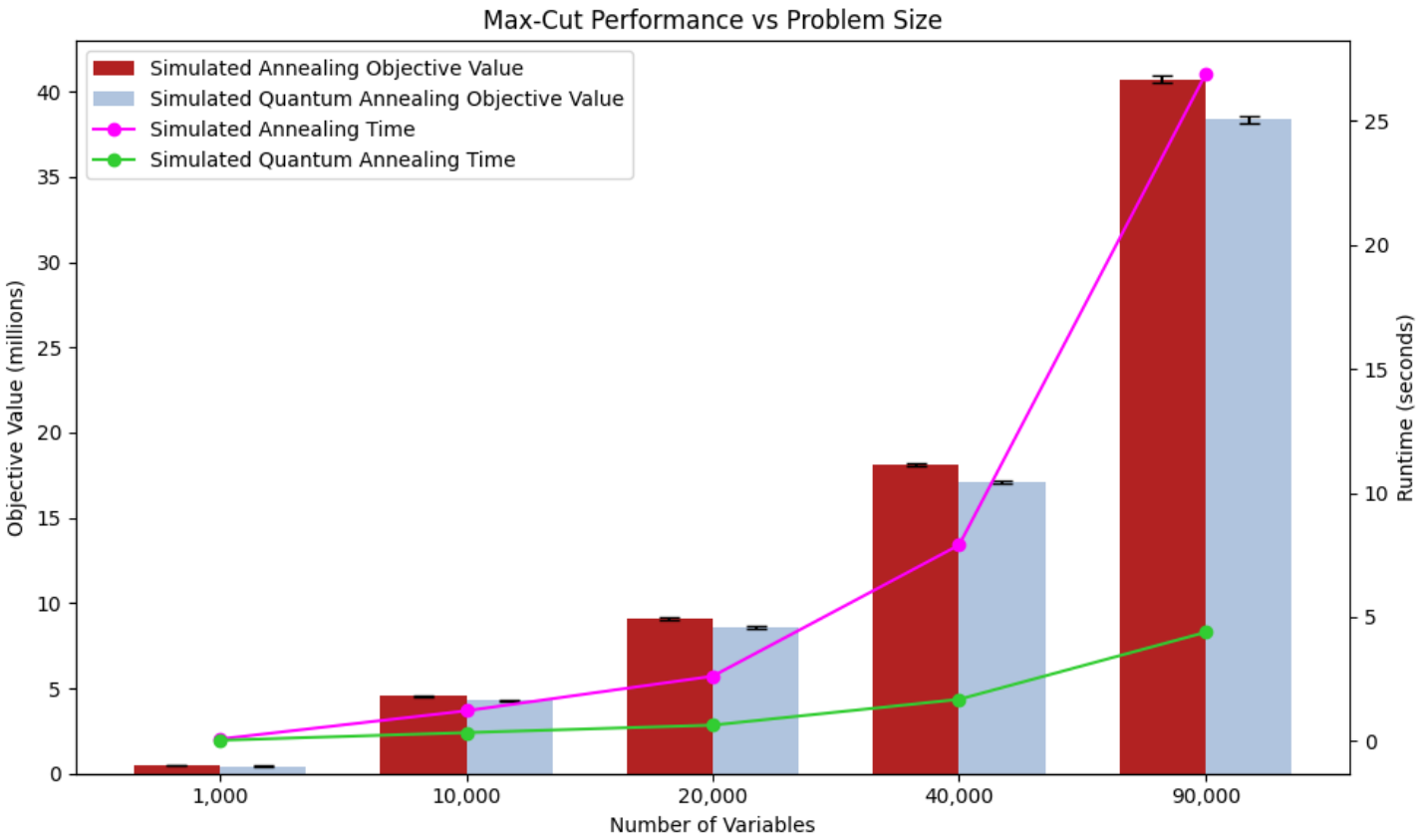

Fig 3.1: Plot of performance results of SA and SQA solvers

3.6 Challenges and Limitations

Despite promising preliminary results, several challenges and limitations remain for the project in the future. One challenge is the performance sensitivity due to parameter selections. Both methods rely on parameters like cooling schedule, annealing length or field strength. Optimal parameter choice may affect the runtimes and solution quality for both methods, and it remains to be seen how much either method may improve from selecting the optimal parameter values.

Limitations arise from the memory usage of large problem instances. As the variable count increases, so does the size of the associated Q matrix. As shown in Table 3.1, the benchmark graphs were highly sparse, thus limiting the number of interconnected variables and the size of the Q matrix. The use of Python dictionaries to represent the Q matrix allowed for any unused connections not have to be stored in system memory, making solution feasible. However, larger and denser problem instances may cause memory issues and place a limit on the size and complexity of the problems we can analyse.

Finally, the results obtained by SQA are classical approximations of quantum annealing. SQA may provide valuable insight into any quantum advantages, but it skips the process of embedding, which can affect performance greatly. Performance gains observed may not directly translate to real quantum annealers.

Chapter 4

Future Work

Moving forward, the scope of benchmarking problems considered will be extended to both portfolio optimisation and job scheduling problems. The portfolio optimisation problem introduces complexities due to how interconnected variables are within the problem encoding. This allows us to investigate how dense QUBO instances are handled by the solvers. Job scheduling introduces complexities, as transforming a given problem into the QUBO framework is often non-trivial. This allows us to investigate how QUBO encoding impacts solver performance.

Data for these problems will then need to be generated or acquired from publicly available sources. QUBO formulations of these problems will then be generated according to the literature. Classical and quantum solvers can then be implemented to solve these problem instances. Finally, the performance of both solvers can be evaluated, and comparisons can be drawn from the results. A timeline for these steps is seen in Fig. 4 .1.

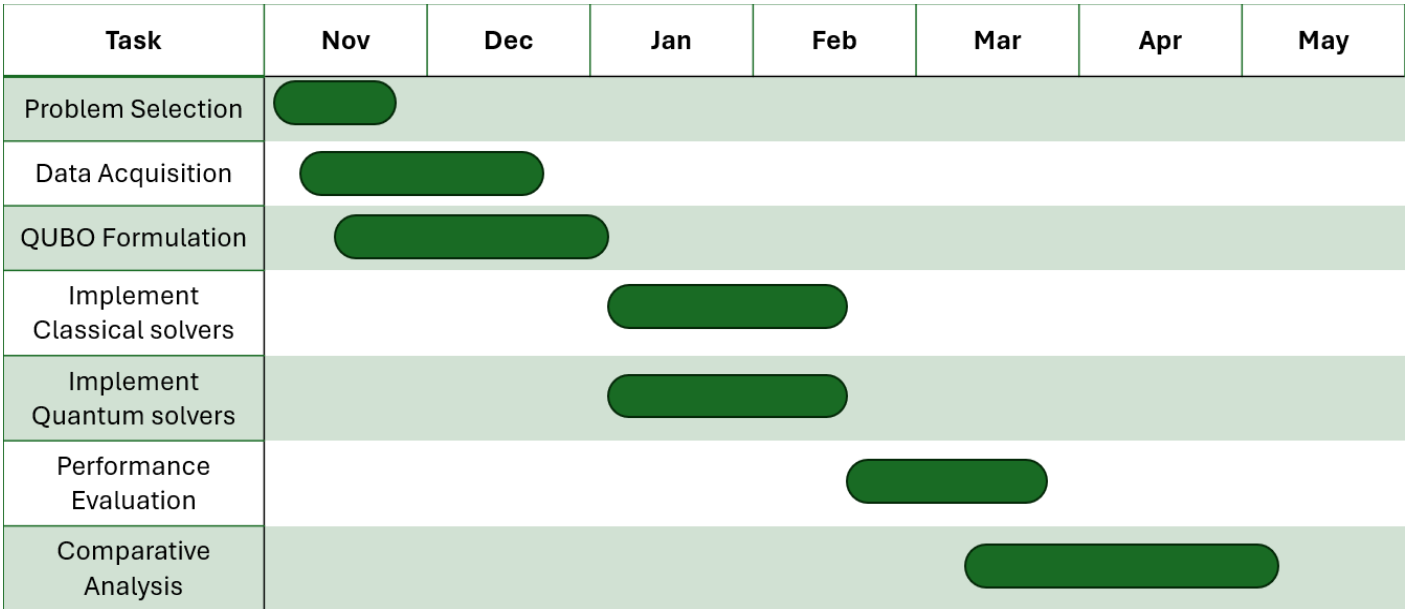

Fig. 4.1 Gantt chart for proposed future work

Chapter 5

Ethics and Sustainability

This project focuses entirely on the benchmarking of classical and quantum optimisation techniques for COPs, which does not involve any personal data or safety systems. However, some ethical considerations are relevant to research integrity and the broader context of emerging technologies.

One ethical issue is that of the transparent and responsible reporting of results. Benchmark studies are sensitive to design choices like problem encoding, solver parameters, and performance metrics considered. Care has and will be taken to use publicly accessible data, consistent QUBO encodings and clear evaluation metrics in order to ensure a fair comparison can be drawn. Overstating the performance of a solver in this case could contribute to misleading claims regarding the advantage of quantum systems. This project focuses on only the measured performance of the solvers and the conclusions that can be drawn from them.

From the perspective of sustainability, complex computational optimisation can draw significant resource costs. Classical solvers may require large amounts of computational time, while current implementations of quantum annealers rely on energy intensive supercooling. The analysis of solver efficiencies and runtimes contributes to an understanding of how COPs may be solved more effectively, potentially reducing resource usage in the future.

References

[1] Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, 2014, doi: https://doi.org/10.3389/fphy.2014.00005.

[2] Vodeb, "Accuracy and Performance Evaluation of Quantum, Classical and Hybrid Solvers for the Max-Cut Problem" 2024, https://arxiv.org/pdf/2412.07460

[3] M. Jünger, G. Reinelt, and G. Rinaldi, *Combinatorial Optimization -- Eureka, You Shrink!* Springer, 2003. https://link.springer.com/chapter/10.1007/3-540-36478-1_17

[4] F. Peres and M. Castelli, "Combinatorial Optimization Problems and Metaheuristics: Review, Challenges, Design, and Development," *Applied Sciences*, vol. 11, no. 14, p. 6449, Jul. 2021, doi: https://doi.org/10.3390/app11146449

[5] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671–680, May 1983, doi: https://doi.org/10.1126/science.220.4598.671.

[6] M.-W. Park and Y.-D. Kim, "A systematic procedure for setting parameters in simulated annealing algorithms," *Computers & Operations Research*, vol. 25, no. 3, pp. 207–217, Mar. 1998, doi: https://doi.org/10.1016/S0305-0548(97)00054-3.

[7] Y. Lin, Z. Bian, and X. Liu, "Developing a dynamic neighborhood structure for an adaptive hybrid simulated annealing – tabu search algorithm to solve the symmetrical traveling salesman problem," *Applied Soft Computing*, vol. 49, pp. 937–952, Dec. 2016, doi: https://doi.org/10.1016/j.asoc.2016.08.036.

[8] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 67–82, Apr. 1997, doi: https://doi.org/10.1109/4235.585893.

[9] T. Kadowaki and H. Nishimori, "Quantum annealing in the transverse Ising model," *Physical Review E*, vol. 58, no. 5, pp. 5355–5363, Nov. 1998, doi: https://doi.org/10.1103/physreve.58.5355.

[10] E. Farhi and J. Goldstone, "A Quantum Adiabatic Evolution Algorithm Applied to Random Instances of an NP-Complete Problem", Apr . 2001, https://arxiv.org/pdf/quant-ph/0104129

[11] S. Kikuura, R. Igata, Y. Shingu and S. Watabe, "Performance Benchmarking of Quantum Algorithms for Hard Combinatorial Optimization Problems: A Comparative Study of non-FTQC Approaches", Oct 2024, https://arxiv.org/pdf/2410.22810

[12] T. F. Rønnow *et al.*, "Defining and detecting quantum speedup," *Science*, vol. 345, no. 6195, pp. 420–424, Jul. 2014, doi: https://doi.org/10.1126/science.1252319.

[13] F. Glover, G. Kochenberger, and Y. Du, "Quantum Bridge Analytics I: a tutorial on formulating and using QUBO models," *4OR*, vol. 17, no. 4, pp. 335–371, Nov. 2019, doi: https://doi.org/10.1007/s10288-019-00424-y.

[14] V. Choi, "Minor-Embedding in Adiabatic Quantum Computation: I. The Parameter Setting Problem", Apr 2008, https://arxiv.org/pdf/0804.4884

[15] H. Wang, "Large-Scale Planar Graph Instances for Max-Cut Problem," *Texas Digital Library (University of Texas)*, Aug. 2019, doi: https://doi.org/10.18738/t8/57q281.

[16] "dwave-samplers — Python documentation," *Dwavequantum.com*, 2025. https://docs.dwavequantum.com/en/latest/ocean/api_ref_samplers/index.html